

MDISC Example 4 (LAPR2 g023)

Tomás Fonseca, Marcelo Oliveira, André Oliveira, Bruno Pinto

Input files (Example 4): `vertices4.csv` (entities, loaded in US19), `edges4.csv` (relations, loaded in US20) and `snapshotdates.csv` (the snapshot dates, used in US32). All the results below were made by the program using these files.

1. Network snapshots (dates from `snapshotdates.csv`)

For each date we count the entities and edges that are active. One entity or edge is active in a date when `startDate <= date <= endDate`.

Date	Active entities	Active edges
2010-01-01	15	8
2014-01-01	21	17
2018-01-01	26	26
2022-01-01	30	34
2026-01-01	30	36

Active entities in each date:

- 2010-01-01: P1 Ana Andrade, P2 Beatriz Bastos, P3 Carlos Cabral, P4 Diogo Dias, P5 Eurico Esteves, P6 Francisca Fontes, C1 Minister of Energy, C3 Judge, C5 Minister of Science and Education, C6 Rector, O1 Ministry of Energy, O3 Green Party, O5 Ministry of Justice, O7 Ministry of Science and Education, O8 Engineering University.
- 2014-01-01: the ones from 2010 plus P7 Gaspar Guedes, P8 Henriqueta Henriques, P9 Ivone Inacio, C2 CEO, O2 EnergyCorp, A2 Company Shares.
- 2018-01-01: the ones from 2014 plus P10 Spike Lee, P11 Jon Snow, C4 Lawyer, O6 Law Firm SA, A1 Company Shares.
- 2022-01-01: the ones from 2018 plus A3 PetShopBoys Company Shares, C7 Administrator, O4 Liberal Party, O9 PetShopBoys.
- 2026-01-01: the same as 2022 (30 entities), but with 2 more active edges (34 to 36).

--- Snapshot: 2010-01-01 ---
Active entities : 15
Active relations: 8

Entities by type:
Person 6
Organization 5
Position 4

Relations by label:
relativeOf 2
inOrganization 4
friendOf 2

--- Snapshot: 2014-01-01 ---
Active entities : 21
Active relations: 17

Entities by type:
Person 9
Organization 6
Position 5
Asset 1

Relations by label:
relativeOf 3
appointedBy 1
holdsPosition 4
inOrganization 4
memberOf 1
friendOf 2
ownerOf 1
associatedWith 1

--- Snapshot: 2018-01-01 ---
Active entities : 26
Active relations: 26

Entities by type:
Person 11
Organization 7
Position 6
Asset 2

Relations by label:
relativeOf 3
appointedBy 2
holdsPosition 6
inOrganization 5
influences 1
ownerOf 2
memberOf 1
friendOf 4
associatedWith 2

--- Snapshot: 2022-01-01 ---
Active entities : 30
Active relations: 34

Entities by type:
Person 11
Organization 9
Position 7
Asset 3

Relations by label:
relativeOf 3
appointedBy 2
holdsPosition 7
inOrganization 6
influences 2
ownerOf 3
controls 1
memberOf 2
friendOf 4
opposes 1
supervises 1
associatedWith 2

--- Snapshot: 2026-01-01 ---
Active entities : 30
Active relations: 36

Entities by type:
Person 11
Organization 9
Position 7
Asset 3

Relations by label:
relativeOf 3
appointedBy 2
holdsPosition 7
inOrganization 6
influences 3
ownerOf 3
controls 2
memberOf 2
friendOf 4
opposes 1
supervises 1
associatedWith 2

The support graph is undirected and not weighted. Every directed edge that is active in 2026-01-01 becomes one link in both directions. The adjacency matrix is 30x30:

	P1	P2	P3	P4	P5	P6	P7	P8	P9	P10	P11	O1	O2	O3	O4	O5	O6	O7	O8	O9	C1	C2	C3	C4	C5	C6	C7	A1	A2	A3		
P1	0	1	0	0	0	0	1	0	0	0	0	0	1	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	1	0	0	
P2	1	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0		
P3	0	1	0	0	0	0	0	0	0	0	0	0	0	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0		
P4	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0		
P5	0	0	0	1	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0		
P6	1	0	0	0	1	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	1	0	0	0	0	0	0	0	1	0		
P7	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0		
P8	0	0	0	0	0	0	1	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0		
P9	0	0	0	0	0	0	0	1	0	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0		
P10	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	1		
P11	0	0	0	0	0	0	0	1	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0		
O1	0	0	0	0	0	0	0	0	0	0	1	0	1	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0		
O2	1	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0		
O3	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0		
O4	0	0	0	1	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0		
O5	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0		
O6	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0		
O7	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0		
O8	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0		
O9	0	0	0	0	0	0	0	0	0	1	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0		
C1	1	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0		
C2	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0		
C3	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0		
C4	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0		
C5	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0		
C6	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0		
C7	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0		
A1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0		
A2	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0		
A3	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0		
=== Support graph at 2026-01-01 (30x30) ===																																
	P1	P2	P3	P4	P5	P6	P7	P8	P9	P10	P11	O1	O2	O3	O4	O5	O6	O7	O8	O9	C1	C2	C3	C4	C5	C6	C7	A1	A2	A3		
P1	0	1	0	0	0	0	1	0	0	0	0	0	1	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	1	0	0	
P2	1	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0		
P3	0	1	0	0	0	0	0	0	0	0	0	0	0	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0		
P4	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0		
P5	0	0	0	1	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0		
P6	1	0	0	0	1	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0		
P7	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0		
P8	0	0	0	0	0	0	1	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0		
P9	0	0	0	0	0	0	0	1	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0		
P10	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1		
P11	0	0	0	0	0	0	0	0	0	1	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0		
O1	0	0	0	0	0	0	0	0	0	0	1	0	1	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0		
O2	1	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0		
O3	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0		
O4	0	0	0	1	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0		
O5	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0		
O6	0	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0		
O7	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0		
O8	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0		
O9	0	0	0	0	0	0	0	0	0	1	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0		
C1	1	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0		
C2	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0		
C3	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0		
C4	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0		
C5	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0		
C6	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0		
C7	0	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0		
A1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0		
A2	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0		
A3	0	0	0	0	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0		

3. US34: Pathway between Beatriz Bastos (P2) and Jon Snow (P11)

Date	Result
2018-01-01	There is no path between Beatriz Bastos and Jon Snow (the two are active, but they are in different parts of the support graph)
2026-01-01	There is a path, the distance is 4 edges

So the result depends on the date. Jon Snow only joins in 2017 and in 2018 he is still in a separated part of the network. In 2026, with the new edges that were created, there is now a path of size 4 until Beatriz. The same two entities give a different answer in each date because of the temporal part.

Checks whether two entities are connected in the support graph and returns the shortest distance (number of edges). Pick a snapshot date to query the network as it stood on that date.

Snapshot date: 

Source entity:

Target entity:

No path found between P2 and P11.

Find Path

Back

Checks whether two entities are connected in the support graph and returns the shortest distance (number of edges). Pick a snapshot date to query the network as it stood on that date.

Snapshot date: 

Source entity:

Target entity:

Pathway verified on the support graph (undirected, unweighted).

Path EXISTS between P2 and P11 — distance: 4 edge(s).

Find Path

Back

4. US34: Procedure (code, pseudocode and complexity)

```

public static int shortestDistance(SupportGraph graph, String sourceId, String targetId) { 20 usages
    if (graph == null) {
        throw new IllegalArgumentException("graph must not be null");
    }

    IndexRegistry registry = graph.getRegistry();

    if (!registry.contains(sourceId)) {
        throw new IllegalArgumentException("unknown source entity: " + sourceId);
    }
    if (!registry.contains(targetId)) {
        throw new IllegalArgumentException("unknown target entity: " + targetId);
    }

    int src = registry.indexFor(sourceId);
    int dst = registry.indexFor(targetId);

    // Trivial case: same entity
    if (src == dst) {
        return 0;
    }

    int n = graph.size();
    boolean[][] adjMatrix = graph.getAdjacencyMatrix();

    // dist[i] = shortest distance from src to i; -1 means not visited
    int[] dist = new int[n];
    for (int i = 0; i < n; i++) {
        dist[i] = NO_PATH;
    }
}

```

```

// BFS queue implemented as a primitive int array (circular buffer)
// Maximum elements ever enqueued = n (each node at most once)
int[] queue = new int[n];
int head = 0;
int tail = 0;

// Initialise BFS from src
dist[src] = 0;
queue[tail] = src;
tail = tail + 1;

while (head != tail) {
    // Dequeue
    int u = queue[head];
    head = head + 1;

    // Explore neighbours using the raw adjacency row
    for (int v = 0; v < n; v++) {
        if (adjMatrix[u][v] && dist[v] == NO_PATH) {
            dist[v] = dist[u] + 1;

            // Early exit: reached the target
            if (v == dst) {
                return dist[v];
            }

            // Enqueue v
            queue[tail] = v;
            tail = tail + 1;
        }
    }
}

return NO_PATH;
}

```

Pseudocode

```

function shortestDistance(graph, source, target):
    if source = target: return 0
    n = number of nodes of the graph
    adj = adjacency matrix of the graph (n x n)
    dist[0..n-1] = -1 // -1 means not visited yet
    queue = empty queue (FIFO)

    dist[source] = 0
    enqueue(queue, source)

    while queue is not empty:
        u = dequeue(queue)
        for v = 0 to n-1:
            if adj[u][v] = 1 and dist[v] = -1: // neighbour not visited

```

```

        dist[v] = dist[u] + 1
        if v == target: return dist[v]          // we reached the target
        enqueue(queue, v)

    return -1                                // there is no path

```

This is a Breadth-First Search (BFS) on the support graph. It visits the nodes by layers, by order of distance to the source, so the first time it reaches the target we already have the shortest path (in number of edges).

Time complexity (worst case)

The code uses an adjacency matrix. Each node enters and leaves the queue at most one time (the check `dist[v] == -1` does not let us visit again), so the while runs at most n times. Inside, the for goes through all the line of the matrix to find the neighbours, which is n operations. So:

$$T(n) = O(n \times n) = O(n^2)$$

where n is the number of entities (nodes) of the support graph. The worst case (source and target with no connection, or target only reached in the end) makes us visit all the nodes and read all the line of each one. Space: $O(n)$ for the `dist` and the queue, plus the matrix $n \times n$.

Note: if we used an adjacency list, the BFS would be $O(n + m)$ (m = number of edges). Because here the graph is a matrix, reading the lines makes it $O(n^2)$.

5. US36: Subnetwork that integrates Beatriz Bastos in 2018-01-01

The connected part of the support graph (in 2018-01-01) that has Beatriz Bastos (P2) has 17 entities.

Members: P1 Ana Andrade, P2 Beatriz Bastos, P3 Carlos Cabral, P4 Diogo Dias, P5 Eurico Esteves, P6 Francisca Fontes, C1 Minister of Energy, C2 CEO, C3 Judge, C4 Lawyer, O1 Ministry of Energy, O2 EnergyCorp, O3 Green Party, O5 Ministry of Justice, O6 Law Firm SA, A1 Company Shares, A2 Company Shares.

This confirms the result of section 3: Jon Snow is not in this subnetwork in 2018, and that is why there is no path between the two in that date.

Extracts the connected subnetwork that contains a chosen entity. All entities in this subnetwork can participate in influence chains that pass through the chosen entity. Pick a snapshot date to extract the subnetwork as it stood on that date.

Snapshot date:

01/01/2018

Origin entity:

P2

Subnetwork of P2: 17 entity(ies).

P1
P2
P3
P4
P5
P6
O1
O2
O3
O5
O6
C1
C2
C3
C4
A1
A2

```

public static SubnetworkResult extract(SupportGraph graph, String originId) {
    if (graph == null) {
        throw new IllegalArgumentException("graph must not be null");
    }
    if (!graph.getRegistry().contains(originId)) {
        throw new IllegalArgumentException("unknown entity id: " + originId);
    }

    int n = graph.size();
    boolean[][] adj = graph.getAdjacencyMatrix();
    int src = graph.getRegistry().indexOf(originId);

    // -----
    // BFS using primitive arrays only (AC2)
    // -----

    boolean[] visited = new boolean[n];
    int[] queue = new int[n];
    int head = 0;
    int tail = 0;

    visited[src] = true;
    queue[tail] = src;
    tail = tail + 1;

    while (head != tail) {
        int u = queue[head];
        head = head + 1;

        for (int v = 0; v < n; v++) {
            if (adj[u][v] && !visited[v]) {
                visited[v] = true;
                queue[tail] = v;
                tail = tail + 1;
            }
        }
    }

    // -----
    // Collect visited node ids into a compact array
    // -----

```

```

// Count visited nodes first (primitive - no list)
int count = 0;
for (int i = 0; i < n; i++) {
    if (visited[i]) {
        count = count + 1;
    }
}

// Map original index → subnetwork index
int[] subIndex = new int[n];
for (int i = 0; i < n; i++) {
    subIndex[i] = -1;
}
String[] subIds = new String[count];
int k = 0;
for (int i = 0; i < n; i++) {
    if (visited[i]) {
        subIndex[i] = k;
        subIds[k] = graph.getRegistry().idAt(i);
        k = k + 1;
    }
}

// -----
// Build the sub-adjacency matrix for visited nodes only
// -----

boolean[][] subAdj = new boolean[count][count];
for (int i = 0; i < n; i++) {
    if (visited[i]) {
        for (int j = 0; j < n; j++) {
            if (visited[j]) {
                subAdj[subIndex[i]][subIndex[j]] = adj[i][j];
            }
        }
    }
}

return new SubnetworkResult(originId, subIds, subAdj);
}

```